

```

UdpEchoServerApplication:UdpEchoServer(0x8a09c0b0)
UdpEchoClientApplication:UdpEchoClient(0x8a09f9f8)
UdpEchoClientApplication:SendDataSize(0x8a09f9f8, 512)
UdpEchoServerApplication:StartApplication(0x8a09c0b0)
UdpEchoClientApplication:StartApplication(0x8a09f9f8)
UdpEchoClientApplication:ScheduleTransmit(0x8a09f9f8, +0.0ns)
UdpEchoClientApplication:Send(0x8a09f9f8)
At time 3s client sent 512 bytes to 192.168.100.2 port 5000
UdpEchoServerApplication:HandleRead(0x8a09c0b0, 0x8a09a2d8)
At time 3.00142s server received 512 bytes from 192.168.100.1 port 49153
Echoing packet
At time 3.00142s server sent 512 bytes to 192.168.100.1 port 49153
UdpEchoClientApplication:HandleRead(0x8a09f9f8, 0x8a09a460)
At time 3.00287s client received 512 bytes from 192.168.100.2 port 5000
UdpEchoServerApplication:StopApplication(0x8a09c0b0)
UdpEchoClientApplication:StopApplication(0x8a09f9f8)
UdpEchoServerApplication:DoDispose(0x8a09c0b0)
UdpEchoServerApplication:DoDispose(0x8a09c0b0)
UdpEchoServerApplication:UdpEchoServer(0x8a09c0b0)
UdpEchoClientApplication:UdpEchoClient(0x8a09f9f8)

```

Figure 1: Output of the Python script

```

[root@deepu ns-3.22]# tcpdump -nn -tt -r pcapsim-0-0.pcap
reading from file pcapsim-0-0.pcap, Link-type PPP (PPP)
3.000000 IP 192.168.100.1.49153 > 192.168.100.2.2000: UDP, Length 512
6.108667 IP 192.168.100.2.2000 > 192.168.100.1.49153: UDP, Length 512
6.000000 IP 192.168.100.1.49153 > 192.168.100.2.2000: UDP, Length 512
6.108667 IP 192.168.100.2.2000 > 192.168.100.1.49153: UDP, Length 512
[root@deepu ns-3.22]# tcpdump -nn -tt -r pcapsim-1-0.pcap
reading from file pcapsim-1-0.pcap, Link-type PPP (PPP)
3.005433 IP 192.168.100.1.49153 > 192.168.100.2.2000: UDP, Length 512
3.005433 IP 192.168.100.2.2000 > 192.168.100.1.49153: UDP, Length 512
6.005433 IP 192.168.100.1.49153 > 192.168.100.2.2000: UDP, Length 512
6.005433 IP 192.168.100.2.2000 > 192.168.100.1.49153: UDP, Length 512

```

Figure 2: Output of tcpdump

```
./waf --pyrun scratch/pythonscript.py
```

Please note that the directory *scratch* is not the default location to save Python scripts and hence it is not mandatory to save Python files in this directory. Also remember to use the path starting from the directory *ns/ns-allinone-3.22/ns-3.22* to the directory containing the Python script. Since I have copied the Python script into the directory *scratch*, the path in the command is *scratch/pythonscript.py*. We have used the option `LOG_LEVEL_ALL` in the simulation script. Thus all the log information is displayed on the terminal. Figure 1 shows the terminal displaying the log information from the Python script.

The Python bindings feature is a work in progress and there are some problems associated with it. One is the insufficient coverage of ns-3 C++ source files. There are many ns-3 APIs yet to be supported by Python bindings. Newly created modules in ns-3 will not be supported by Python bindings. A tool called *pybindgen* is used to generate Python bindings for ns-3 source code written in C++. If you feel that all this fuss about the use of Python in ns-3 is unnecessary, then you can simply disable Python in ns-3. Open a terminal in the directory *ns/ns-allinone-3.22/ns-3.22* and execute the following command:

```
./waf configure --disable-python
```

On execution of this command, Python will be disabled in ns-3, and you can continue executing ns-3 simulation with the help of C++ simulation scripts as discussed in the previous parts of this series.

Tracing with pcap

We have earlier discussed ASCII based tracing in ns-3. But the ASCII trace file format is not the only one available in ns-3. It is possible to obtain a pcap trace file from an ns-3 simulation. An application programming interface for capturing network traffic is called pcap. A clear advantage of pcap tracing over ASCII tracing is that pcap trace files need not be processed manually. A lot of information can be obtained from pcap trace files with the help of utilities like tcpdump, Wireshark, etc.

If pcap tracing is such a treasure trove of information, why did I hide it till the end? Well, for the simple reason that pcap trace files are processed by third-party utilities. It took me a long time to start discussing ns-3 in this series. So then, I thought it was not the right time to discuss pcap tracing and introduce yet another utility. But since we are about to wind up this series, I now believe the time is ripe to learn pcap tracing. In Linux, pcap is implemented in the *libpcap* library. To discuss the features of pcap tracing I will again use the simulation script *simulation1.cc* with which we are quite familiar. I have renamed the simulation script as *pcapsim.cc*. The only significant change made in the script *pcapsim.cc* is the addition of the following line of code:

```
pointToPoint.EnablePcapAll("pcapsim");
```

It enables pcap tracing in the ns-3 script. The extension of pcap trace files is *.pcap*. But the name of the pcap trace file generated is not *pcapsim.pcap* because naming conventions for pcap trace files and ASCII trace files are different. Unlike ASCII tracing, multiple trace files are generated in pcap tracing depending on the number of nodes in the simulation and the number of devices installed on these nodes. For example, if the simulation contains four nodes with each node having three devices installed on them, then a total of 12 pcap trace files will be generated. Thus, the number of pcap trace files generated equals the total number of devices in the simulation. In pcap tracing, the name given inside the function *EnablePcapAll()* in double quotes is just a suffix for the different pcap trace files generated. The name of each pcap trace file will be as follows: *file_name_suffix-node_number-device_number.pcap*.

In the simulation script *pcapsim.cc* there are only two nodes, and each has just one device installed on it. So, on execution, the simulation script will generate two pcap trace files named *pcapsim-0-0.pcap* and *pcapsim-1-0.pcap*. The execution of the simulation script *pcapsim.cc* is no different from the execution of all the other C++ simulation scripts in ns-3. Copy the file *pcapsim.cc* in the directory *scratch* and open a terminal in the directory *ns/ns-allinone-3.22/ns-3.22*. Execute the commands `./waf` and `./waf --run pcapsim` on the terminal. This will generate the two pcap trace files mentioned above. By default, the